Inventory Scanning Roadmap

Architecture Principle

```
barcode → API → transaction → updated stock
```

Devices do not matter. Input methods do not matter. Everything is just: **barcode → POST /api/scan → stock movement → updated level**.

Build this pipeline once, and the system scales without redesign.

Current State (What Already Exists)

Component	Status	Location
`products` table with `barcode` index	☑ Built	`src/db/schema.ts`
`warehouses` table (= branches)	☑ Built	`src/db/schema.ts`
`inventory` table (cached stock levels)	☑ Built	`src/db/schema.ts`
`inventory_transactions` (audit ledger)	☑ Built	`src/db/schema.ts`
`users` table with RBAC	☑ Built	`src/db/schema.ts`
`adjustStock()` / `transferStock()`	☑ Built	`src/services/inventory.ts`
Document scanner agent (GPT-4o vision)	☑ Built	`src/agent/document-scanner/`
Document ingestion pipeline (5-layer dedup)	☑ Built	`src/services/document-ingestion.ts`
Image-based scan endpoints (`/scan/barcode`, `/scan/stock`, `/scan/invoice`)	☑ Built	`src/api/scanning.ts`

Phase 1 — Core Scan Pipeline (Foundation) ☑ BUILT

New Database Tables

Table	Purpose	Location

Table	Purpose	Location
`scan_events`	Raw log of every scan attempt (success/fail/pending)	`src/db/schema.ts`
`idempotency_keys`	Prevents duplicate stock changes from retries/offline sync	`src/db/schema.ts`

Schema Changes

- `inventory_transactions.device_type` column added (varchar, nullable)
  - Tracks: `web`, `mobile`, `scanner`, `api`

Unified Scan Endpoint

POST /api/scan

```
{
  "barcode": "8901234567890",
  "warehouseId": "uuid",
  "deviceType": "web",
  "quantity": 1,
  "scanType": "scan_add",
  "notes": "Received from supplier",
  "idempotencyKey": "client-uuid-123"
}
```

Response:

```
{
  "data": {
    "success": true,
    "scanEventId": "uuid",
    "transactionId": "uuid",
    "product": {
      "id": "uuid",
      "name": "Widget A",
      "sku": "WDG-001",
      "barcode": "8901234567890",
      "unit": "piece"
    },
    "previousStock": 50,
    "newStock": 51,
    "quantityChanged": 1,
    "warehouseId": "uuid",
    "scanType": "scan_add",
    "deviceType": "web",
    "timestamp": "2026-02-23T10:30:00Z"
  }
}
```

Atomic Pipeline Flow

```
1. Check idempotency key (return cached response if duplicate)
2. Insert scan_events record (raw log — always, even if product missing)
3. Lookup product by barcode
4. Duplicate detection (same barcode < 2 seconds)
5. Validate stock levels (for removals)
6. Insert inventory_transactions row
7. Atomically update inventory (upsert)
8. Link transaction → scan event
9. Cache response for idempotency
10. Return updated stock level
```

Additional Endpoints

Method	Path	Purpose
`POST`	`/api/scan`	Process single barcode scan
`POST`	`/api/scan/batch`	Process offline queue (array of scans)
`GET`	`/api/scan/lookup/:code`	Instant product lookup by barcode
`GET`	`/api/scan/history`	Filterable scan event log

Files Created/Modified

File	Action
`src/db/schema.ts`	Added `scanEvents`, `idempotencyKeys` tables + relations; added `deviceType` to `inventoryTransactions`
`src/services/scan.ts`	NEW — `processScan()`, `processBatchScan()`, `lookupBarcode()`, `getScanHistory()`, `cleanupExpiredIdempotencyKeys()`
`src/api/scan.ts`	NEW — REST endpoints for scan pipeline
`src/api/index.ts`	Added barrel export for scan routes

Critical Rules

1. **`inventory` = cache, `inventory_transactions` = truth.** If inventory is rebuilt from transactions, numbers must match.
2. **Every scan creates a `scan_events` record** — even failures. This enables audit, fraud detection, debugging.
3. **Never update stock without a transaction row.** The ledger is immutable.
4. **Idempotency keys prevent duplicates.** Network drops, retries, rapid double-taps are all safe.

Phase 2 — Web App Scanner Support ☑ BUILT

2a. Scanner-as-Keyboard Input (USB/Bluetooth scanners) ☑

Library: `onscan.js@1.5.2` (MIT, 9.7K weekly downloads)

- Framework-agnostic keyboard-wedge detection
- Distinguishes rapid scanner input from human typing via timing analysis
- Config: `suffixKeyCodes: [13]` (Enter), `reactToPaste: true`, `minLength: 4`, `avgTimeByChar: 50`
- Dynamically imported in ScanPage.tsx useEffect
- Reports `deviceType: "usb_scanner"` to API when hardware scanner detected
- Auto-ignores when INPUT/TEXTAREA/SELECT elements are focused
- Visual "🔗 Hardware scanner detected" badge with pulse animation

This supports:

- USB barcode scanners (keyboard HID)
- Bluetooth scanners paired to device
- Warehouse handheld scanners (Zebra, Honeywell)
- POS workflow quick-add

2b. Camera Scanning in Browser ☑

Library: `barcode-detector@3.0.8` (MIT, 54.8K weekly downloads)

- W3C BarcodeDetector polyfill using ZXing-C++ WebAssembly
- Imported as ponyfill: `import { BarcodeDetector } from "barcode-detector/ponyfill"`
- Works in ALL browsers (Safari, Firefox, Chrome, Edge) — no native API dependency
- Supports: codabar, code_39, code_93, code_128, databar, ean_8, ean_13, itf, upc_a, upc_e, aztec, data_matrix, pdf417, qr_code, micro_qr_code

Frontend features built:

- "📷 Scan" camera toggle in mode selector
- `getUserMedia()` camera permission request with rear-camera preference
- Live video preview with overlay
- Continuous barcode detection loop via `detector.detect(video)`
- Auto-submit detected barcode to `POST /api/scan`
- Manual barcode entry fallback mode
- Visual confirmation: ☑ success flash + product name, ✕ error toast

Libraries evaluated and rejected:

- `dynamsoft-javascript-barcode` — DEPRECATED, requires paid license
- `html5-qrcode` — 3 years unmaintained, stale
- `@nicolo-ribaudo/barcode-reader` — superseded by barcode-detector polyfill

Phase 3 — Real-Time Feedback ☑ BUILT

3a. Live Stock Updates After Scan ☑

Implementation: Server-Sent Events (SSE)

Backend (`src/api/scan.ts`):

- `GET /api/scan/events` SSE endpoint using `sse()` middleware from `@agentuity/runtime`
- `scanStreams` Set tracks connected SSE clients
- `broadcastStockUpdate()` pushes events to all clients after successful scans
- 20-second keepalive pings prevent connection drops
- `stream.onAbort()` cleanup removes disconnected clients
- Same SSE pattern as chat.ts (proven infrastructure)

Event payload:

```
{
  "type": "stock_update",
  "properties": {
    "productId": "uuid",
    "productName": "Widget A",
    "barcode": "8901234567890",
    "warehouseId": "uuid",
    "warehouseName": "",
    "previousStock": 50,
    "newStock": 51,
    "scanType": "scan_add",
    "userId": "uuid"
  }
}
```

Frontend (`src/web/pages/ScanPage.tsx`):

- EventSource connecting to `/api/scan/events`
- Exponential backoff reconnect (1s → 30s max)
- Parses `stock_update` events and shows info flash notifications
- Auto-reconnects on connection loss

3b. Implementation Choice

Approach	Complexity	Decision
Polling every 5s	Low	❌ Rejected — unnecessary load
SSE (Server-Sent Events)	Medium	☑ **Chosen** — reuses existing SSE infra
WebSocket	Higher	☐ Future — for live dashboards

Phase 4 — Offline Resilience ☑ BUILT

4a. Offline Scan Queue ☑

Implementation: IndexedDB (no Service Worker needed)

- Database: `biq-scan-queue`, object store: `pending-scans`, keyed by `idempotencyKey`

- Helpers: `openOfflineDB()`, `enqueueOfflineScan()`, `getOfflineScans()`, `clearOfflineScans()`, `removeOfflineScan()`
- Detects `navigator.onLine` state + `online`/`offline` event listeners
- **Offline:** Stores scans to IndexedDB, shows "📴 Queued offline" flash
- **Network errors during online scan:** Auto-falls back to offline queue
- **On reconnect:** Auto-calls `syncOfflineScans()` via `online` event listener
- **Manual sync:** "⚡ Sync Now" button for user-triggered sync
- **Batch upload:** Sends queued scans via `POST /api/scan/batch`
- **Per-item cleanup:** Removes successful items from IndexedDB individually
- **UI indicators:**
  - 🔴 Red status bar when offline
  - 🟡 Yellow status bar when scans pending
  - Queue count badge
  - Syncing progress state

4b. Conflict Resolution ☑

- Idempotency keys prevent duplicate stock changes (same key = cached response)
- Server returns cached response for retried keys
- Each scan is tagged with a client timestamp for ordering
- Offline scans include device type and all original metadata

---

Phase 5 — Validation & Stability ☑ BUILT

5a. Server-Side Validation ☑

- ☑ Barcode format validation (string, 1-255 chars)
- ☑ UUID validation on warehouseId
- ☑ Quantity validation (integer, min 1)
- ☑ Device type enum validation
- ☑ Rate limiting per endpoint

5b. Duplicate Detection ☑

- ☑ Same barcode + same warehouse + same user within 2 seconds = rejected
- ☑ Idempotency key checked before processing
- ☑ Request hash comparison for key reuse with different payloads

5c. Hardware Scanner Device Support ☑

Library: `onscan.js@1.5.2` + `@types/onscan.js@1.5.6`

- ☑ USB barcode scanner detection via keyboard-wedge timing analysis
- ☑ Bluetooth scanner support (same HID protocol)
- ☑ Device type reported as `usb_scanner` in scan API
- ☑ Auto-submit on scan detection (no manual Enter needed)
- ☑ Visual hardware scanner badge with pulse animation

5d. Future Enhancements ☐

- Configurable duplicate window (env var `SCAN_DEDUP_WINDOW_MS`)
- Barcode format validation (EAN-13, UPC-A, Code-128 pattern matching)
- Admin-configurable scan rate limits
- Scan anomaly detection (unusual patterns → admin alert)

Phase 6 — Future Expansion ☐ NOT STARTED

These reuse the same `POST /api/scan` pipeline:

- Native Android app (Kotlin → same API)
- Native iOS app (Swift → same API)
- Warehouse automation (PLC/conveyor → API integration)
- Supplier integrations (EDI → scan events)
- AI demand prediction (uses scan_events + inventory_transactions history)
- Multi-barcode per product (junction table: `product_barcodes`)
- Batch/lot tracking (add `lot_number`, `expiry_date` to scan_events)
- Unit conversion engine (kg ↔ g, crate ↔ piece)

No pipeline redesign needed — the architecture scales.

MVP Definition of Done

- ☑ **Schema:** `scan_events` + `idempotency_keys` tables created
- ☑ **Service:** `processScan()` atomic pipeline with full error handling
- ☑ **API:** `POST /api/scan` endpoint with auth + rate limiting
- ☑ **API:** `POST /api/scan/batch` for offline sync
- ☑ **API:** `GET /api/scan/lookup/:code` for barcode → product lookup
- ☑ **API:** `GET /api/scan/history` for audit trail
- ☑ **API:** `GET /api/scan/events` SSE real-time stock updates
- ☑ **Migration:** Tables created and live (56 tables confirmed)
- ☑ **Frontend:** Scan page with mode selector (Camera, Manual, Lookup, History)
- ☑ **Frontend:** Camera scanning with `barcode-detector` WASM polyfill (all browsers)
- ☑ **Frontend:** Manual barcode entry with Enter-key submit
- ☑ **Frontend:** Visual confirmation (success/error flash)
- ☑ **Frontend:** Hardware scanner support via `onscan.js`
- ☑ **Frontend:** Offline scan queue (IndexedDB + auto-sync)
- ☑ **Frontend:** Connectivity status bar + sync button
- ☑ **Frontend:** SSE client for real-time stock change notifications
- ☑ **Frontend:** Sidebar navigation + RBAC (staff, manager, admin)
- ☐ **Test:** USB scanner → stock updates correctly
- ☐ **Test:** Phone camera → stock updates correctly
- ☐ **Test:** Offline queue → syncs on reconnect
- ☐ **Test:** Duplicate scan within 2s → rejected
- ☐ **Test:** Same idempotency key → cached response returned

---

Database Schema (New Tables)

scan_events

```
id               uuid  PK
warehouse_id     uuid  FK → warehouses.id
user_id          uuid  FK → users.id
barcode          varchar(255)  NOT NULL, indexed
device_type      varchar(30)   NOT NULL  DEFAULT 'web'
status           varchar(30)   NOT NULL  DEFAULT 'pending_sync'
linked_transaction_id  uuid   nullable
product_id       uuid  FK → products.id, nullable
quantity         integer  NOT NULL  DEFAULT 1
scan_type        varchar(30)  NOT NULL  DEFAULT 'scan_add'
error_message    text  nullable
idempotency_key varchar(100)  unique, nullable
raw_payload      jsonb  nullable
metadata         jsonb  nullable
created_at       timestamptz  NOT NULL  DEFAULT now()
```

idempotency_keys

```
id               uuid  PK
key              varchar(255)  NOT NULL, unique
request_hash     varchar(64)   NOT NULL
response_snapshot  jsonb  NOT NULL
expires_at       timestamptz  NOT NULL
created_at       timestamptz  NOT NULL  DEFAULT now()
```

inventory_transactions (modified)

```
+ device_type   varchar(30)  nullable  -- NEW column
```